

API перехвата команд МК-61

Версия документа: 29.07.2026

Назначение

Command-hook API позволяет расширению эмулятора перехватить внешнюю команду МК-61 по ее байту opcode. Команда может быть введена с клавиатуры или прочитана из программы. Для нее можно:

- выполнить callback перед штатным исполнением;
- заменить opcode только для текущего исполнения;
- получить callback после завершения команды;
- изменить вычисленный результат, например X после КИП7;
- одновременно зарегистрировать несколько opcode и несколько callback для одного opcode.

Таблицы ROM и байты программы при этом не изменяются. Для работы с внутренними адресами K145ИК существует отдельный низкоуровневый ROM-hook API, описанный в MK61s-mini-ROM-Hooks.md.

Два уровня перехвата

Важно не смешивать внешний opcode и адрес командного слова микросхемы:

Уровень	Пример	API
Команда пользователя	D7 = КИП7, 3В = К СЧ	MK-61 command hook
Внутреннее состояние ROM	IK1302:06, IK1306:A7	ROM command hook

Для перехвата КИП7 регистрируется opcode 0xD7. Адрес A7 для этого не нужен. В документации ГСЧ запись IK1306:A7 означает внутреннюю точку другой микросхемы, где К СЧ уже исполняется и временное слово x_i находится в нужной позиции ST.

Публичные типы

Интерфейс объявлен в code/mk61emu_core.h:

```
enum class Mk61CommandHookPhase : u8 {
    BEFORE_EXECUTE,
    AFTER_EXECUTE
};

enum class Mk61CommandSource : u8 {
    KEYBOARD,
    PROGRAM
};

struct Mk61CommandHookContext {
    Mk61CommandHookPhase phase;
    Mk61CommandSource source;
    u8 opcode;
    u8 replacement_opcode;
    u32 sequence;
};

using Mk61CommandHook =
    void (*)(Mk61CommandHookContext& context, void* user_data);

using Mk61CommandHookHandle = u32;
```

Mk61CommandHookHandle идентифицирует одну регистрацию. Значение INVALID_MK61_COMMAND_HOOK равно нулю и означает ошибку регистрации.

Функции API

```

Mk61CommandHookHandle register_mk61_command_hook(
    u8 opcode,
    Mk61CommandHookPhase phase,
    Mk61CommandHook callback,
    void* user_data = nullptr);

bool unregister_mk61_command_hook(Mk61CommandHookHandle handle);

usize registered_mk61_command_hook_count();

```

Регистрация

Одна регистрация задает один opcode, одну фазу и один callback. Чтобы получать обе фазы одной команды, нужно зарегистрировать два hook.

Функция возвращает INVALID_MK61_COMMAND_HOOK, если:

- callback равен nullptr;
- значение phase недопустимо;
- заполнены все шестнадцать пользовательских слотов;
- регистрация выполняется из уже работающего command callback.

user_data передается без копирования. Вызывающая сторона должна сохранять объект действительным до удаления регистрации.

Удаление

unregister_mk61_command_hook() удаляет только одну регистрацию. Остальные hook того же или другого opcode продолжают работать.

В handle хранится поколение слота. Поэтому устаревший или уже удаленный handle не может удалить новую регистрацию, позднее занявшую тот же слот.

Счетчик

registered_mk61_command_hook_count() возвращает число пользовательских регистраций. Встроенный hook ГСЧ использует отдельный зарезервированный слот и в счетчик не входит.

Контекст callback

Поле	Смысл
phase	Текущая фаза BEFORE_EXECUTE или AFTER_EXECUTE
source	Источник команды: KEYBOARD или PROGRAM
opcode	Исходный байт команды, которую выдал пользователь или программа
replacement_opcode	Байт, реально переданный штатному декодеру
sequence	Идентификатор текущего перехваченного исполнения

Одна команда получает одинаковые source, opcode и sequence в обеих фазах. В AFTER_EXECUTE поле replacement_opcode показывает, что реально было исполнено после всей цепочки BEFORE_EXECUTE.

Только replacement_opcode в фазе BEFORE_EXECUTE является управляющим полем. Записи в остальные поля контекста игнорируются. В AFTER_EXECUTE изменение replacement_opcode уже ничего не меняет.

Регистрация нескольких команд

Для каждой команды возвращается независимый handle:

```
Mk61CommandHookHandle hooks[] = {
    register_mk61_command_hook(
        0xD7, Mk61CommandHookPhase::BEFORE_EXECUTE,
        before_kip7, data7),
    register_mk61_command_hook(
        0xD8, Mk61CommandHookPhase::BEFORE_EXECUTE,
        before_kip8, data8),
    register_mk61_command_hook(
        0x3B, Mk61CommandHookPhase::AFTER_EXECUTE,
        after_random, random_data)
};
```

Один callback также можно зарегистрировать для нескольких opcode с разным user_data. Удаляется каждый handle отдельно.

Шестнадцать слотов — это число регистраций, а не число уникальных opcode. Например, фазы BEFORE и AFTER для восьми команд занимают все шестнадцать слотов. Именно так M61 реализует до восьми одновременных bind.

Несколько callback одной команды

Пользовательские callback одного opcode + phase выполняются в порядке регистрации и получают общий replacement_opcode текущего исполнения.

```
void first(Mk61CommandHookContext& context, void*) {
    context.replacement_opcode = 0xD8;
}

void second(Mk61CommandHookContext& context, void*) {
    // Здесь replacement_opcode уже равен D8.
}
```

Оба callback были зарегистрированы для исходного D7. После замены публичный диспетчер не запускает заново callback, зарегистрированные для D8. Это предотвращает рекурсию и сохраняет детерминированный порядок.

Подмена команды

В BEFORE_EXECUTE можно заменить команду для одного исполнения:

```
void replace_kip7_with_kip8(
    Mk61CommandHookContext& context, void*) {
    context.replacement_opcode = 0xD8;
}
```

При исполнении программного или клавиатурного КИП7 штатный декодер получит D8. Исходный байт программы, ROM и следующий вызов КИП7 не меняются.

Чтобы подавить обычную однобайтную команду и выполнить свою логику в AFTER, ее можно заменить на MK61_NOP, то есть 0x54.

Команды с операндом

Opcode 51, 53, 57-5E занимают в программе два байта. API заменяет только первый opcode и не исправляет длину программы. Если двухбайтную команду заменить на однобайтную или наоборот, дальнейшая интерпретация байта-операнда будет определяться уже новой штатной командой. Расширение обязано учитывать это само.

Подмена X после КИП7

КИП7 имеет opcode D7. В фазе AFTER штатная команда уже:

- обработала косвенный адрес;
- прочитала выбранный регистр памяти;
- поместила результат в X.

Поэтому callback может заменить только итоговый X, не вмешиваясь во внутренний ST:

```

void after_kip7(Mk61CommandHookContext& context, void*) {
    if(context.phase != Mk61CommandHookPhase::AFTER_EXECUTE)
        return;

    const char digits[8] = {'4','2','4','2','4','2','4','2'};
    write_stack_register(stack:X, ' ', digits, 0);
}

Mk61CommandHookHandle hook = register_mk61_command_hook(
    0xD7,
    Mk61CommandHookPhase::AFTER_EXECUTE,
    after_kip7);

```

После КИП7 пользователь увидит 4.2424242. Такой callback работает и для клавиатурной команды, и для D7 в программе. Исходный регистр-указатель и память меняются только так, как предписывает штатный КИП7.

Косвенные команды КПn и КИПn

Внешние opcode распределены последовательно:

Команда	Opcode
КПО ... КПЕ	B0 ... BE
КИПО ... КИПЕ	D0 ... DE

Для регистра n правило изменения указателя штатным микрокодом различается:

Регистры	Действие перед обращением
0 ... 3	Предварительное уменьшение значения регистра
4 ... 6	Предварительное увеличение значения регистра
7 ... E	Значение регистра используется без изменения

Command hook это различие видит корректно, потому что перехватывает общий внешний opcode до входа в штатный алгоритм, а AFTER вызывается после его завершения. Отдельно распознавать внутренние пути для 0-3, 4-6 и 7-E расширению не требуется.

Например, callback AFTER(D7) видит X уже загруженным из регистра памяти, номер которого находился в R7. Для D2 callback увидит также уже уменьшенное штатным алгоритмом значение R2.

Где находятся точки перехвата

Внешний opcode собран в тетрадах IK1302:

```
opcode = R[30] + 16 * R[33]
```

Проверены две точки, где значение уже готово, но штатная команда еще не начала исполнение:

Источник	Точка декодера
Команда из программы	IK1302:06
Команда с клавиатуры	IK1302:97

В BEFORE итоговый replacement_opcode записывается обратно в эти две тетрады до чтения командного слова ROM. Это временная подмена текущего исполнения.

Для программы достижение следующего IK1302:06 одновременно означает, что предыдущая команда завершилась. Поэтому порядок такой:

```

AFTER предыдущей команды
BEFORE следующей команды
штатное исполнение следующей команды

```

Для C/P в конце программы AFTER вызывается при возврате ядра из режима RUN. Для клавиатуры ядро отмечает возврат в штатный экранный цикл IK1302:33, а сам callback AFTER выполняет на границе

`core_61::step()`. В этой точке публичные функции записи логических регистров безопасны.

Связь с К СЧ МК61s

ГСЧ является встроенным клиентом command-hook API:

- 1 Внутренний BEFORE hook зарегистрирован для внешнего opcode 3B.
- 2 После пользовательской цепочки он проверяет итоговый `replacement_opcode`.
- 3 Если реально будет исполнен 3B, устанавливается одноразовый флаг.
- 4 Низкоуровневый ROM hook `IK1306:A7` видит флаг и подставляет новое `xi`.
- 5 Штатный микрокод К СЧ вычисляет результат и помещает его в `X`.

Поэтому публичная замена 3B -> 54 не расходует поток ГСЧ, а замена другой команды на 3B включает тот же усиленный seed. Адрес A7 не распознает пользовательскую команду - он используется только как точная точка доступа к транзитному слову `xi`.

Ограничения callback

Callback выполняется синхронно. Во время него нельзя:

- регистрировать или удалять command hook;
- рекурсивно вызывать `core_61::step()`, `core_61::enable()` или другой путь, запускающий ядро;
- сохранять ссылку на контекст и использовать ее после возврата;
- выполнять долгую блокирующую работу.

API не выделяет динамическую память, не владеет `user_data` и не вызывает его деструктор.

Регистрации сохраняются при `core_61::enable()` и сбросе калькулятора. Сбрасывается только незавершенное текущее исполнение. Поэтому владелец расширения должен явно удалить ненужные handle.

Быстрый путь

Наличие hook для каждого из 256 opcode хранится в общей 32-байтовой битовой карте. Отдельные карты BEFORE и AFTER не нужны: горячему декодеру важно только понять, требуется ли для opcode создавать состояние перехваченной команды, а точная фаза проверяется в слотах уже при полном dispatch.

В прошивках STM32F401/F411 биты карты читаются и изменяются через SRAM bit-band alias Cortex-M4. Обычная команда без hook проверяется одной индексированной загрузкой `LDR.W`, без сдвига и маски. Установка и снятие бита выполняются атомарной записью alias-слова. Реализация написана на C++ без ручного ассемблера; настольные тесты используют эквивалентный масочный путь.

Проверки

Тесты `tests/mk_math_self_test.cpp` проверяют:

- разные opcode и несколько callback одного opcode;
- порядок регистрации и передачу `replacement_opcode` по цепочке;
- отсутствие повторного dispatch после `D7 -> D8`;
- одинаковую замену `D7 -> D8` с клавиатуры и из программы;
- замену `X` в AFTER(`D7`) для клавиатурного и программного КИП7;
- завершение клавиатурной команды без явного `SetKeyPress(0, 0)`;
- последовательные команды, выполненные внутри одного `core_61::step()`;
- AFTER команды С/Р в конце программы;
- независимое удаление, устаревшие handle и емкость реестра;

- взаимодействие замен 3B -> NOP и D7 -> 3B с усиленным ГСЧ.